

Here's a **complete, structured list of conditional expressions** used with **Target Typing + Lambda Expressions in Java (Java 8 → 21)**, along with **clear code examples**.

✓ 1. What is a Conditional Expression in Java?

A **conditional expression** uses the **ternary operator** (**? :**):

condition ? expression1 : expression2;

- In **lambda + target typing**, Java infers the type based on context (functional interface).
-

✓ 2. Conditional Expressions with Lambda (Target Typing)

📌 Case 1: Basic Lambda with Conditional Expression

```
interface MyFunc {  
    int apply(int x);  
}  
  
public class Test {  
    public static void main(String[] args) {  
        MyFunc func = (x) -> (x > 0) ? x : -x; // absolute value  
  
        System.out.println(func.apply(-10)); // 10  
    }  
}
```

- **Target Type** = MyFunc
 - Compiler infers lambda return type = int
-

❏ Case 2: Conditional Returning Different Values

```
interface Checker {  
    String check(int x);  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Checker c = (x) -> (x % 2 == 0) ? "Even" : "Odd";  
  
        System.out.println(c.check(5)); // Odd  
    }  
}
```

❏ Case 3: Nested Conditional Expression

```
interface Grade {  
    String getGrade(int marks);  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Grade g = (m) ->  
            (m >= 90) ? "A" :  
            (m >= 75) ? "B" :  
            (m >= 50) ? "C" : "Fail";  
  
        System.out.println(g.getGrade(80)); // B  
    }  
}
```

❏ Case 4: Conditional with Different Return Types (Type Inference)

```
interface Func {  
    Object apply(int x);  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Func f = (x) -> (x > 0) ? x : "Negative";  
  
        System.out.println(f.apply(-5)); // Negative  
    }  
}
```

□ Compiler finds **common supertype = Object**

▣ Case 5: Conditional Expression with Functional Interface Return

```
import java.util.function.Function;  
  
public class Test {  
    public static void main(String[] args) {  
  
        Function<Integer, Integer> func =  
            (x) -> (x > 0) ? x * 2 : x * -2;  
  
        System.out.println(func.apply(-5)); // 10  
    }  
}
```

▣ Case 6: Conditional Lambda Assignment (Target Typing)

```
interface Operation {  
    int operate(int a, int b);  
}  
  
public class Test {  
    public static void main(String[] args) {
```

```

    boolean add = true;

    Operation op = add
        ? (a, b) -> a + b
        : (a, b) -> a * b;

    System.out.println(op.operate(2, 3)); // 5
}
}

```

□ Both branches must match **same functional interface**

❏ Case 7: Conditional with Method Reference

```

import java.util.function.Function;

public class Test {
    public static void main(String[] args) {

        boolean flag = true;

        Function<String, Integer> func =
            flag ? String::length : String::hashCode;

        System.out.println(func.apply("Hello"));
    }
}

```

❏ Case 8: Conditional with Void Lambda

```

interface Printer {
    void print(String msg);
}

```

```

public class Test {
    public static void main(String[] args) {

        boolean upper = true;

        Printer p = upper
            ? (msg) -> System.out.println(msg.toUpperCase())
            : (msg) -> System.out.println(msg.toLowerCase());

        p.print("Hello");
    }
}

```

🔗 Case 9: Conditional Inside Lambda Block

```

interface Calculator {
    int compute(int x, int y);
}

public class Test {
    public static void main(String[] args) {

        Calculator calc = (x, y) -> {
            return (x > y) ? x - y : y - x;
        };

        System.out.println(calc.compute(10, 20)); // 10
    }
}

```

🔗 Case 10: Conditional with Generics + Target Typing

```

import java.util.function.Predicate;

public class Test {
    public static void main(String[] args) {

```

```

    Predicate<Integer> p = (x) -> (x > 10) ? true : false;

    System.out.println(p.test(15)); // true
}
}

```

✓ 3. Summary Table

Case	Expression Type	Target Type Role	Key Concept
1	Simple ternary	Infers return type	Basic lambda
2	String return	Functional interface match	Type consistency
3	Nested ternary	Multi-branch inference	Readability concern
4	Mixed types	Common supertype	Object inference
5	Built-in FI	Function<T,R>	Standard API
6	Lambda selection	Same FI required	Compile-time check
7	Method reference	Target typing resolves	Clean syntax
8	Void lambda	Consumer-like	No return
9	Block lambda	Explicit return	Complex logic
10	Generics	Predicate	Boolean result

✔ 4. Key Rules (Very Important)

✔ Rule 1: Same Target Type Required

Both branches of ?: must match:

```
condition ? lambda1 : lambda2;
```

✔ Rule 2: Compatible Return Types

```
(x) -> (x > 0) ? 10 : 10.5; // becomes double
```

✔ Rule 3: Target Typing Drives Inference

Without target type → ✗ compilation error

```
var f = (x) -> (x > 0) ? x : -x; // ✗ ambiguous
```

✔ Rule 4: Functional Interface Context is Mandatory

```
Function<Integer, Integer> f = ...
```

✔ 5. Interview-Level Insight

- Conditional expressions in lambdas rely heavily on:
 - **Target typing**
 - **Type inference rules**
 - **Common supertype resolution**
- Java compiler uses:
 - **Poly expressions**

- **Context-sensitive typing**

If you want, I can next give:

- ✓ Fail cases (compilation errors)
- ✓ Internal compiler inference logic (JLS deep dive)
- ✓ Real-world examples (Spring / Streams / APIs)

Here's a **complete evolution of iteration techniques in Java Collections (Java 1 → Java 21)** presented in a clean table with **code examples and explanations**.

✓ **Java Collection Iteration Methods (Java 1 → Java 21)**

Java Version	Iteration Method	Code Example	Description
Java 1.0	Enumeration	<pre>java Vector<String> v = new Vector<>(); Enumeration<String> e = v.elements(); while(e.hasMoreElements()){ System.out.println(e.nextElement()); }</pre>	Legacy method used with Vector, Hashtable.
Java 1.2	Iterator	<pre>java List<String> list = new ArrayList<>(); Iterator<String> it = list.iterator(); while(it.hasNext()){ System.out.println(it.next()); }</pre>	Standard iteration introduced in Collections Framework.
Java 1.2	ListIterator	<pre>java List<String> list = new ArrayList<>(); ListIterator<String> it = list.listIterator(); while(it.hasNext()){ System.out.println(it.next()); }</pre>	Supports bidirectional traversal.
Java 1.5	Enhanced for-loop	<pre>java List<String> list = new ArrayList<>(); for(String s : list){</pre>	Simplifies iteration

(Java 5)	(for-each)	System.out.println(s); }	using syntactic sugar.
Java 1.8	forEach() (Iterable)	java list.forEach(s -> System.out.println(s));	Internal iteration using lambda expressions.
Java 1.8	Stream API	java list.stream().forEach(s -> System.out.println(s));	Functional-style iteration.
Java 1.8	Parallel Stream	java list.parallelStream().forEach(s -> System.out.println(s));	Multi-threaded iteration using ForkJoinPool.
Java 1.8	forEach with Method Reference	java list.forEach(System.out::println);	Cleaner lambda replacement.
Java 1.8	Splititerator	java Splititerator<String> sp = list.splititerator(); sp.forEachRemaining(System.out::println);	Supports parallel iteration and splitting.
Java 1.8	Map forEach	java map.forEach((k,v) -> System.out.println(k + " " + v));	Direct iteration over key-value pairs.
Java 9	Stream takeWhile	java list.stream().takeWhile(s -> s.length() < 5).forEach(System.out::println);	Conditional iteration (short-circuiting).
Java 9	Stream dropWhile	java list.stream().dropWhile(s -> s.length() < 5).forEach(System.out::println);	Skips elements based on condition.
Java 10+	var with loop	java for(var s : list){ System.out.println(s); }	Type inference in iteration.
Java 16+	Stream.toList() iteration	java list.stream().toList().forEach(System.out::println);	Immutable list iteration.

Java 21	SequencedCollection (new API)	java SequencedCollection<String> sc = new ArrayList<>(); sc.forEach(System.out::println);	Supports ordered iteration (first/last elements).
Java 21	Record + Stream iteration	java record Person(String name){} List<Person> list = List.of(new Person("A")); list.stream().forEach(p -> System.out.println(p.name()));	Modern style with records.
All Versions	Classic for-loop (index-based)	java List<String> list = new ArrayList<>(); for(int i=0;i<list.size();i++){ System.out.println(list.get(i)); }	Useful for indexed access.

❑ Key Differences (Quick Understanding)

Type	Style	Example
External Iteration	Manual control	Iterator, for-loop
Internal Iteration	Library controlled	forEach, Streams
Sequential	One thread	stream()
Parallel	Multi-threaded	parallelStream()

❑ Best Practice (Modern Java)

❑ Preferred in **Java 8+**

list.forEach(System.out::println);

❑ For processing:

```
list.stream()  
  .filter(s -> s.startsWith("A"))  
  .forEach(System.out::println);
```

❑ For performance:

```
list.parallelStream().forEach(System.out::println);
```

❑ Important Notes

- **Enumeration** → obsolete (avoid)
 - **Iterator** → still useful for remove operations
 - **forEach / Streams** → most preferred today
 - **Parallel Stream** → use carefully (thread safety)
 - **Splititerator** → advanced use cases (parallel frameworks)
-

If you want, I can also provide:

- ✓ Internal working of each method
- ✓ Fail-fast vs fail-safe iteration (deep dive)
- ✓ Interview questions on iteration
- ✓ Performance comparison (benchmark table)

Here's a **deep dive into Fail-Fast vs Fail-Safe iteration in Java Collections** with clear explanations, internal working, and code examples.

❑ Fail-Fast vs Fail-Safe Iterators (Deep Dive)

Feature	Fail-Fast Iterator	Fail-Safe Iterator
Definition	Immediately throws exception if collection is structurally modified	Works on a copy; no exception
Exception	ConcurrentModificationException	No exception
Collection Types	ArrayList, HashMap, HashSet	ConcurrentHashMap, CopyOnWriteArrayList
Iteration Type	Direct on original collection	Iterates over cloned snapshot
Thread Safety	Not thread-safe	Thread-safe
Performance	Faster	Slower (due to copy)
Modification Allowed	Only via Iterator.remove()	Allowed anytime
Memory Usage	Low	Higher (copy created)

❑ 1. Fail-Fast Iterator (Internal Working)

🔗 Concept

Fail-fast iterators use a **modification count (modCount)** to detect changes.

- Each collection maintains a modCount
- Iterator stores expectedModCount
- If mismatch → exception thrown

✔ Example (Fail-Fast)

```
import java.util.*;
```

```
public class FailFastExample {  
    public static void main(String[] args) {  
        List<String> list = new ArrayList<>();  
        list.add("A");  
        list.add("B");  
  
        Iterator<String> it = list.iterator();  
  
        while (it.hasNext()) {  
            String value = it.next();  
            System.out.println(value);  
  
            // Structural modification  
            list.add("C"); // ✗ Causes exception  
        }  
    }  
}
```

✗ Output

A
Exception in thread "main" java.util.ConcurrentModificationException

❓ Why Exception Happens?

```
if (modCount != expectedModCount)  
    throw new ConcurrentModificationException();
```

☐ Modification outside iterator breaks consistency.

✓ Safe Way (Fail-Fast)

```
Iterator<String> it = list.iterator();
```

```
while (it.hasNext()) {
    String value = it.next();
    if (value.equals("A")) {
        it.remove(); // ✓Safe
    }
}
```

❑ 2. Fail-Safe Iterator (Internal Working)

🔗 Concept

Fail-safe iterators operate on a **clone/snapshot** of the collection.

- No modCount checking
 - Changes don't affect iteration
-

✓ Example (Fail-Safe)

```
import java.util.concurrent.*;

public class FailSafeExample {
    public static void main(String[] args) {
        CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();

        list.add("A");
        list.add("B");

        for (String s : list) {
            System.out.println(s);

            list.add("C"); // ✓No exception
        }
    }
}
```

```
}  
}
```

✓ Output

A
B

☐ Newly added "C" is **not iterated** (snapshot behavior)

⚡ Internal Comparison (Core Logic)

Fail-Fast (Simplified)

```
class Iterator {  
    int expectedModCount = modCount;  
  
    void next() {  
        if (modCount != expectedModCount)  
            throw new ConcurrentModificationException();  
    }  
}
```

Fail-Safe (Simplified)

```
class Iterator {  
    Object[] snapshot = collection.clone();  
  
    void next() {  
        // iterate over snapshot  
    }  
}
```

❑ Real-Time Analogy

Type	Analogy
Fail-Fast	Reading a live document → changes detected immediately
Fail-Safe	Reading a photocopy → original changes don't matter

❑ When to Use What?

✔ Use Fail-Fast

- Single-threaded applications
 - Performance-critical systems
 - No concurrent modification needed
-

✔ Use Fail-Safe

- Multi-threaded applications
 - Concurrent read/write required
 - Avoid exceptions
-

❑ Important Interview Points

- Fail-fast is **best effort**, not guaranteed
- ConcurrentModificationException is **runtime exception**
- Fail-safe is not officially termed in Java docs (but widely used)
- Iterator.remove() is safe in fail-fast

□ Summary

- **Fail-Fast** → Detects & fails immediately
 - **Fail-Safe** → Works on copy, no failure
 - Choose based on **performance vs concurrency needs**
-

If you want next:

- ✓ Deep dive into modCount with JDK source code
- ✓ Parallel stream vs iterator thread behavior
- ✓ Interview tricky MCQs on fail-fast vs fail-safe

Here's a **deep, version-wise explanation of how Java internally stores objects in collections (Java 1 → Java 21)** with architecture, memory model, and code-level behavior.

□ Internal Logic of Storing Objects in Java Collections (1 → 21)

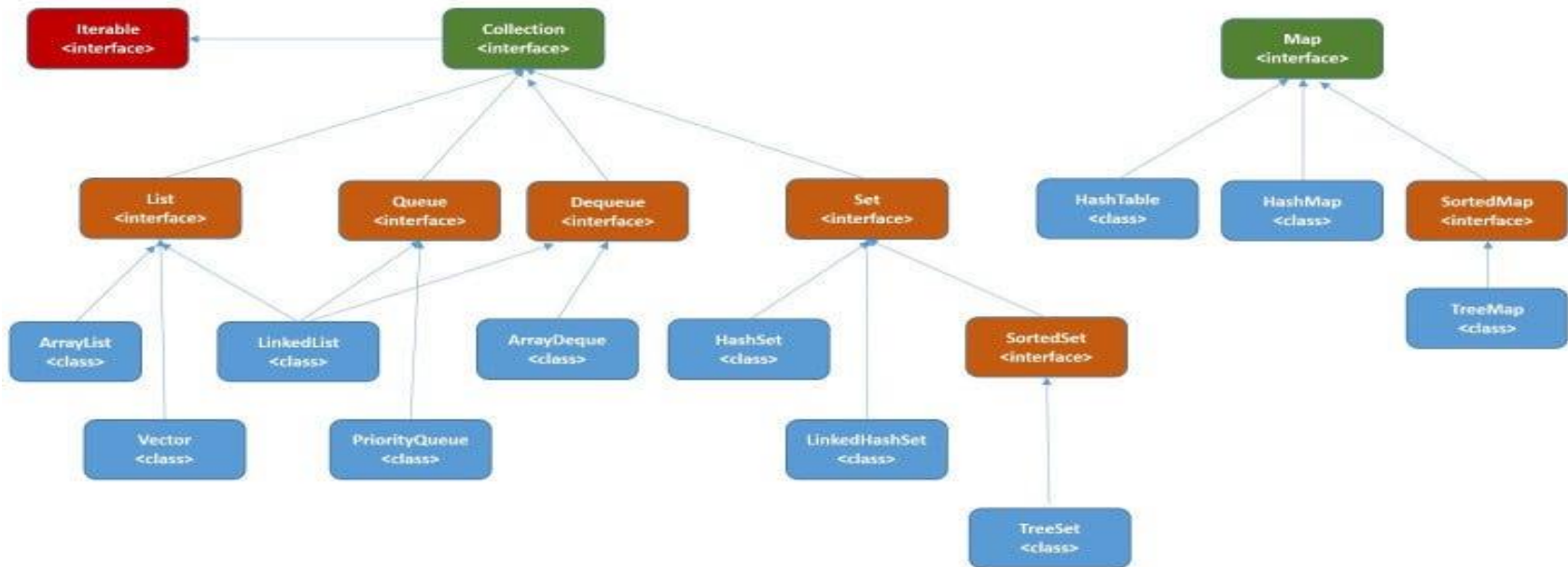
📊 Version-wise Evolution Table

Java Version	Collection Type	Internal Storage Logic	Core Mechanism
Java 1.0	Vector, Hashtable	Array-based + synchronized	Uses Object[]
Java 1.2	ArrayList	Dynamic array	Resize (grow by 1.5x)
Java 1.2	LinkedList	Doubly linked list	Node(prev, next, data)
Java 1.2	HashMap	Array + linked list	Hashing + buckets

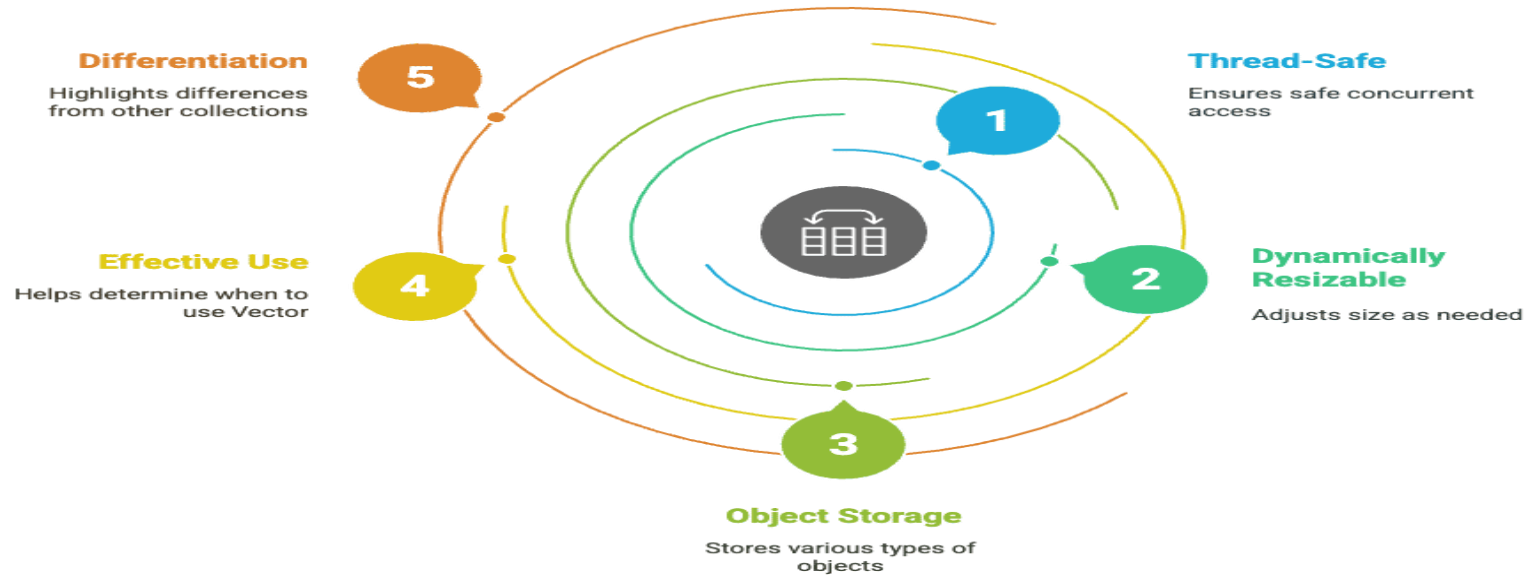
Java 1.4	LinkedHashMap	HashMap + linked list	Maintains insertion order
Java 1.5	Generics	Type-safe storage	Compile-time checks
Java 1.7	HashMap optimization	Bucket collision handling	Linked list per bucket
Java 1.8	HashMap Treeify	LinkedList → Red-Black Tree	O(log n) lookup
Java 8	ConcurrentHashMap	Segment-less locking	CAS + Node array
Java 9	Immutable Collections	Compact storage	Fixed-size arrays
Java 10+	var + inference	Same storage	Syntax improvement
Java 16+	Records	Immutable object storage	Compact object model
Java 21	SequencedCollection	Ordered access	First/Last operations

□ **1. Array-Based Storage (Vector, ArrayList)**

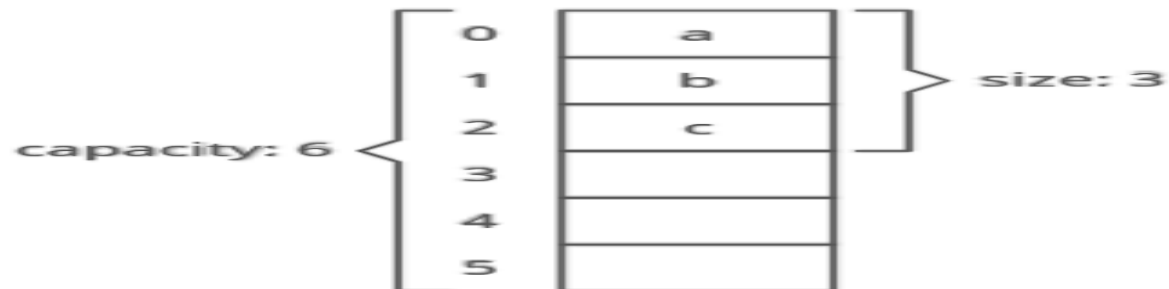
Collection Framework Hierarchy



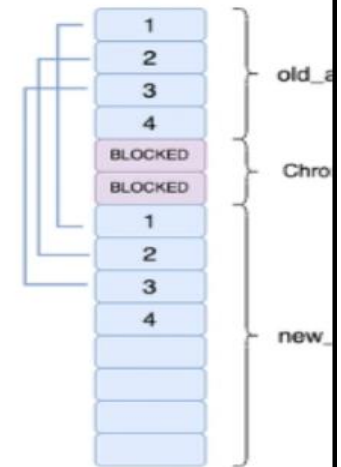
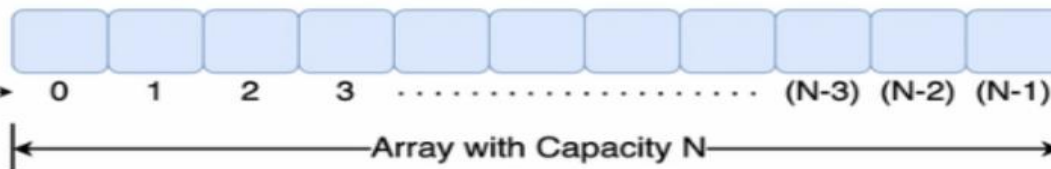
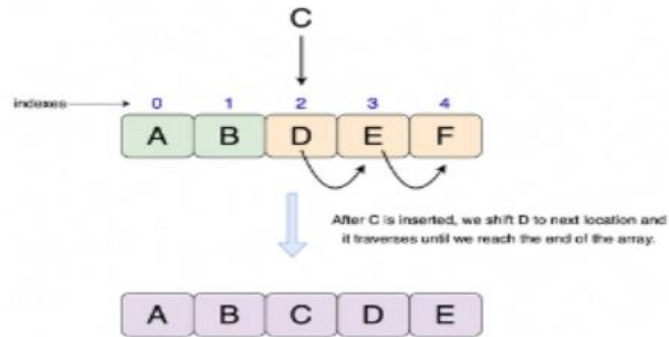
Understanding Java's Vector Class



Made with  Napkin



Dynamic Arrays vs Traditional Arrays



Internal Logic

```
Object[] elementData;  
int size;
```

How it Works

1. Objects stored in **contiguous memory (array)**
2. When full → new array created

3. Old elements copied

🔗 Resize Formula

$\text{newCapacity} = \text{oldCapacity} + (\text{oldCapacity} \gg 1); // 1.5x \text{ growth}$

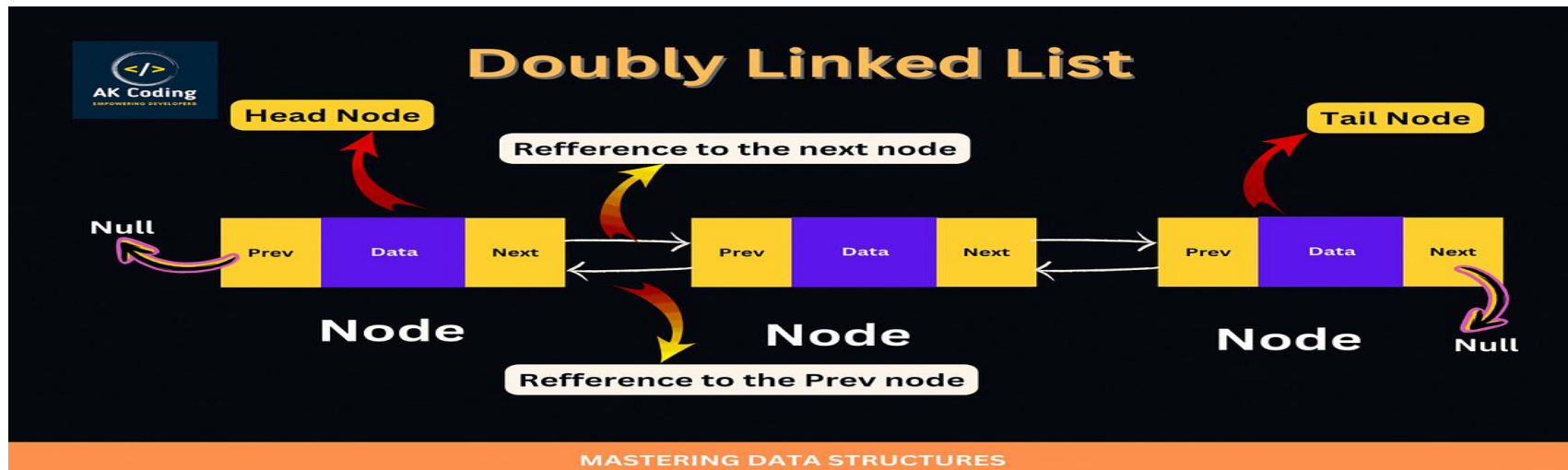
✓ Example

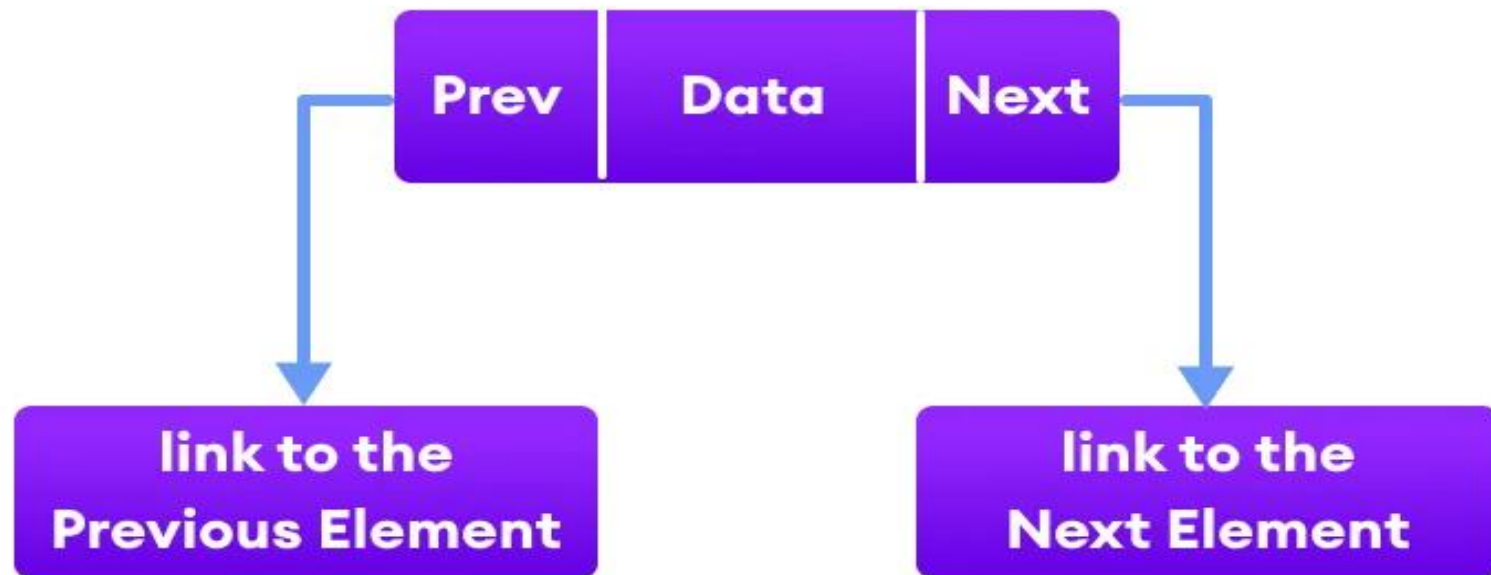
```
List<String> list = new ArrayList<>();  
list.add("A"); // stored at index 0  
list.add("B"); // stored at index 1
```

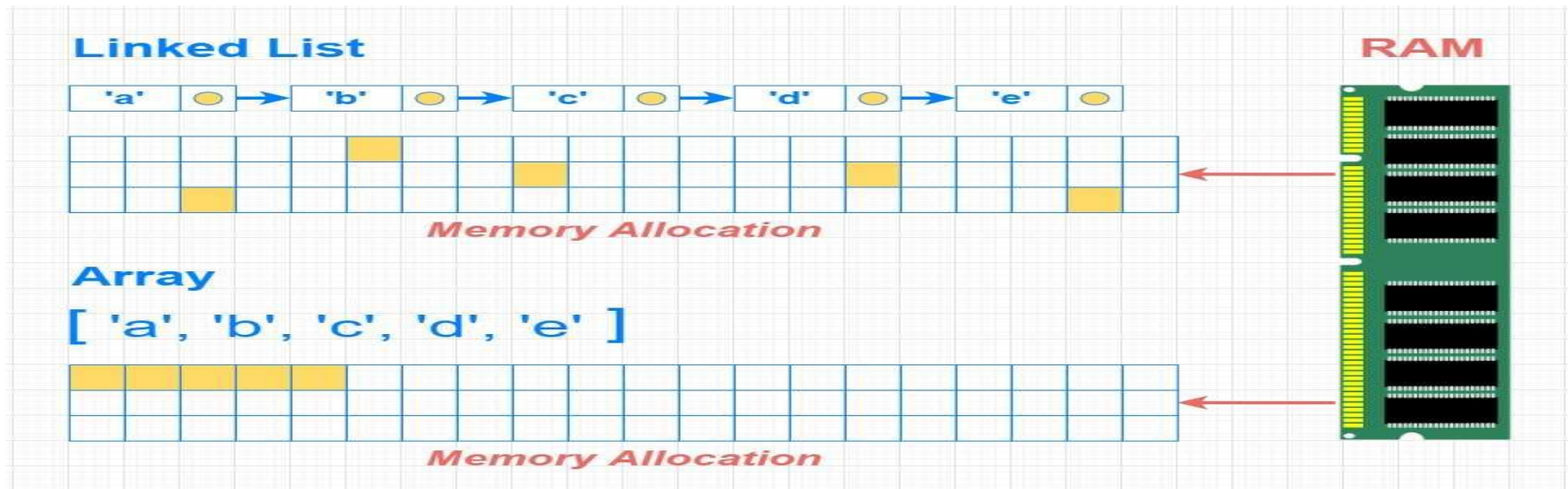
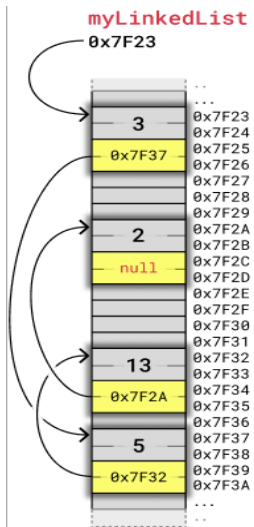
□ Internally:

[A, B, null, null, ...]

□ 2. Linked Storage (LinkedList)







❓ Internal Node Structure

```
class Node<E> {  
    E item;  
    Node<E> next;  
    Node<E> prev;  
}
```

❓ How it Works

- Each object stored in a **node**
- Nodes connected via pointers

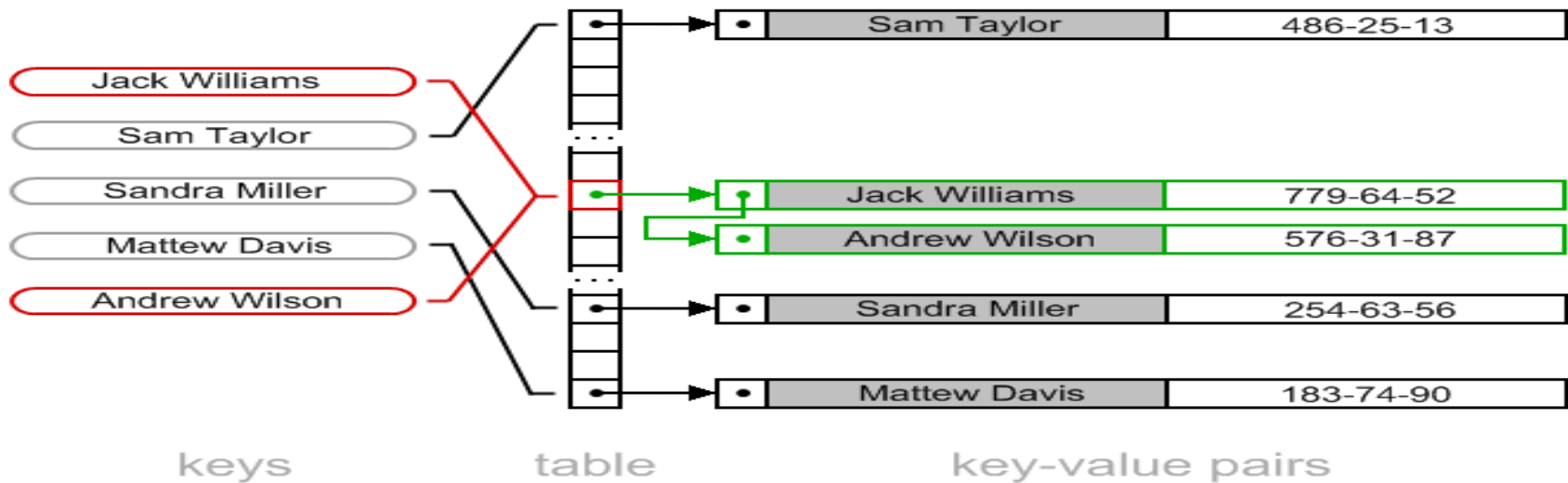
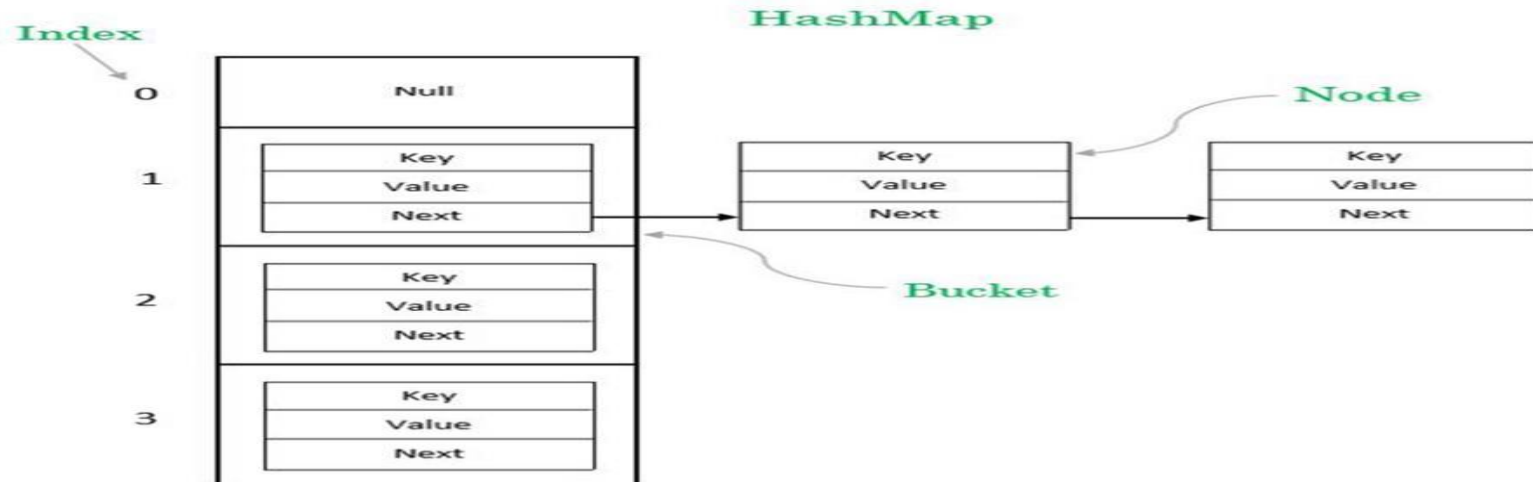
✓ Example

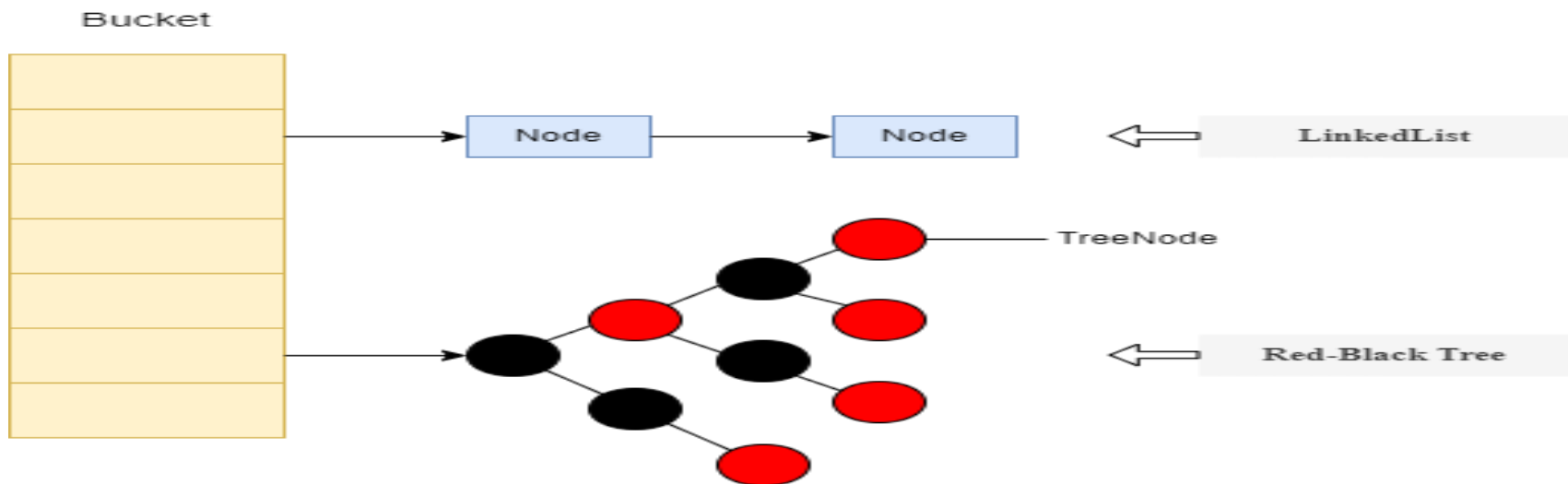
```
LinkedList<String> list = new LinkedList<>();  
list.add("A");  
list.add("B");
```

□ Internally:

```
null <- [A] <-> [B] -> null
```

□ 3. Hash-Based Storage (HashMap)

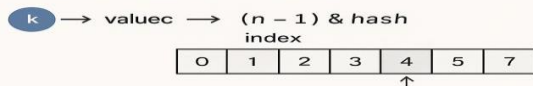




HOW HASHMAP WORKS IN JAVA

HASHING & BUCKETS

When you store a key-value pair, Java computes a hash to determine the bucket



🔄 equals() – THE SAME KEY?

Even with a hash collision, Java uses equals() to check key equality



🌳 FROM LINKED LIST TO RED-BLACK TREE

After too many nodes in a bucket, Java converts it to a Red-Black Tree



❓ Internal Structure

`Node<K,V>[] table;`

Each bucket:

```
class Node<K,V> {  
    int hash;  
    K key;  
    V value;  
    Node<K,V> next;  
}
```

❓ How Object is Stored

1. Compute hash:

```
int hash = key.hashCode();
```

2. Find index:

```
index = hash & (n - 1);
```

3. Store in bucket:

- Empty → direct insert
 - Collision → linked list / tree
-

❓ Java 8 Optimization

If bucket size > 8:

LinkedList → Red-Black Tree

✓ Example

```
Map<Integer, String> map = new HashMap<>();  
map.put(1, "A");  
map.put(17, "B"); // collision possible
```

❑ 4. Ordered Hash Storage (LinkedHashMap)

🔍 Combines:

- HashMap + Doubly Linked List

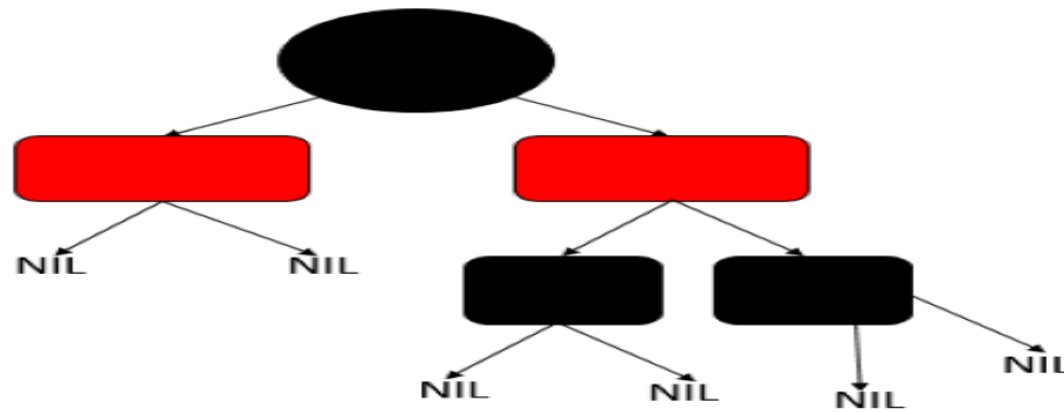
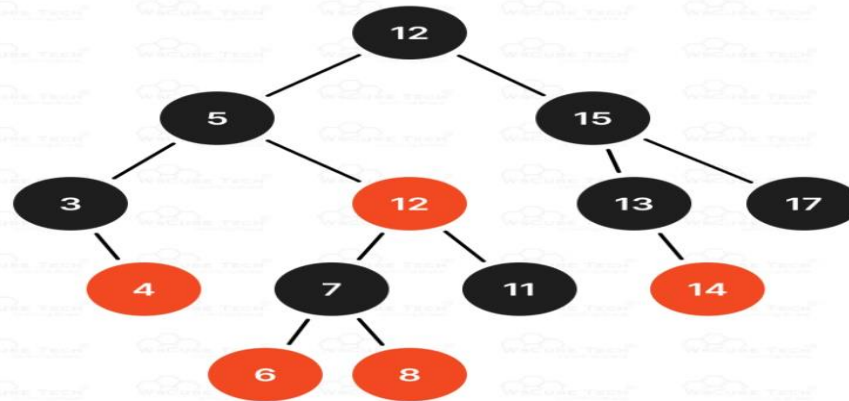
Entry<K,V> before, after;

❑ Maintains:

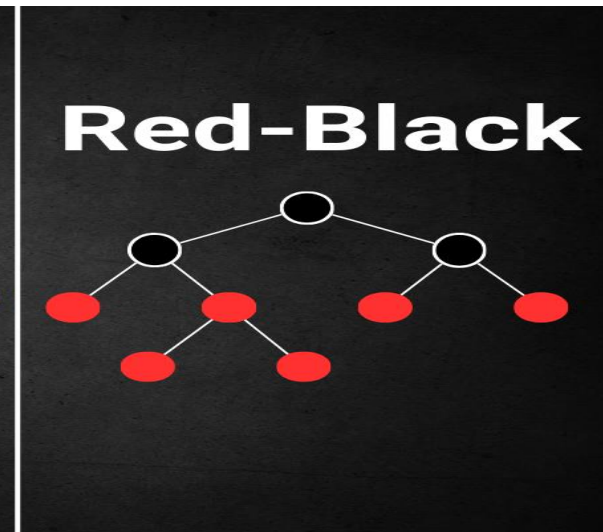
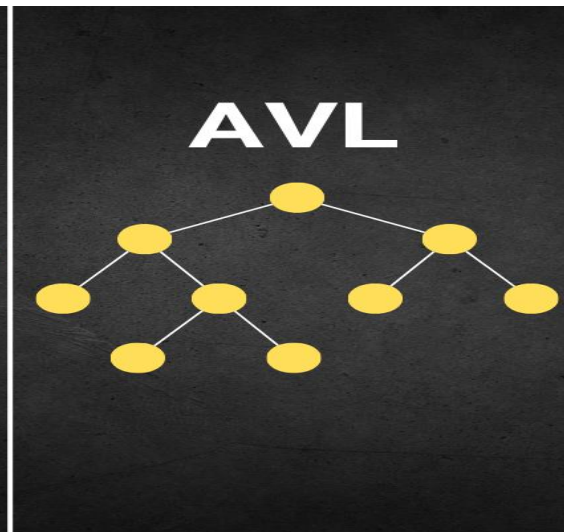
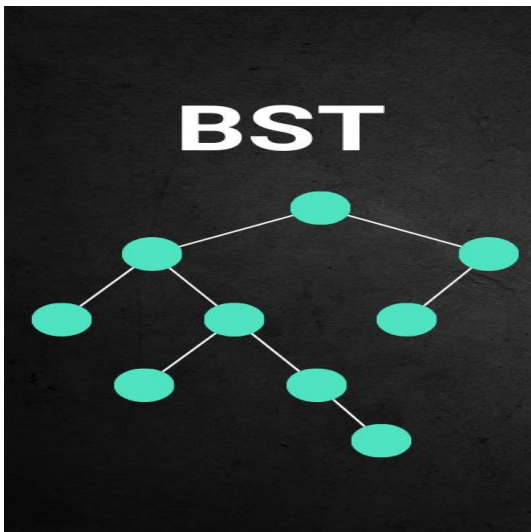
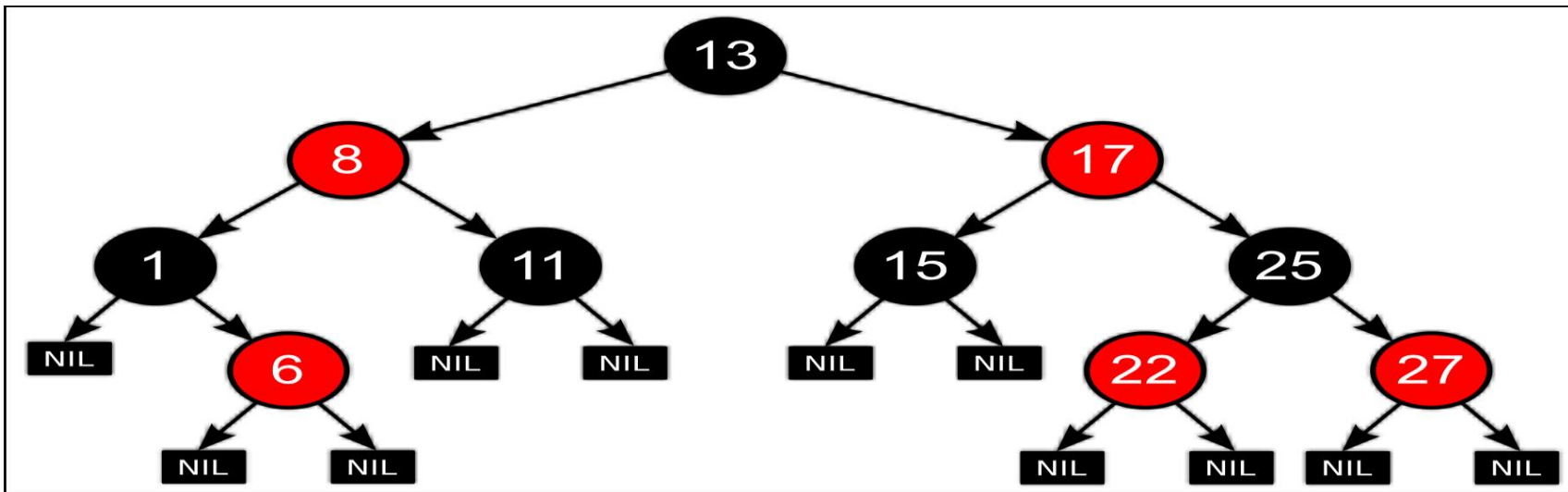
- Insertion order OR access order
-

❑ 5. Tree-Based Storage (TreeMap / TreeSet)

Red Black Tree



Structure of Red Black Tree



🔗 Internal Structure

Entry<K,V> root;

🔗 Logic

- Self-balancing **Red-Black Tree**
- Sorted storage

✓ Example

```
TreeMap<Integer, String> map = new TreeMap<>();  
map.put(2, "B");  
map.put(1, "A");
```

☐ Internally sorted:

1 → A

2 → B

☐ 6. Concurrent Storage (ConcurrentHashMap)

🔗 Java 7 vs Java 8+

Version Mechanism

Java 7 Segmented locking

Java 8+ CAS + synchronized bins

☒ Logic

- No full locking
 - Multiple threads modify safely
-

☐ 7. Immutable Collections (Java 9+)

```
List<String> list = List.of("A", "B");
```

☒ Internal Logic

- Fixed-size array
 - No resize, no modification
-

☐ 8. Modern Enhancements (Java 16-21)

☒ Records

```
record Person(String name) {}
```

- Compact immutable objects
-

☒ SequencedCollection (Java 21)

```
SequencedCollection<String> sc = new ArrayList<>();  
sc.getFirst();  
sc.getLast();
```

☐ End-to-End Flow (Example: HashMap Put)

```
map.put("key", "value");
```

Internal Steps:

1. hashCode() called
2. Hash spread (reduce collisions)
3. Index calculated
4. Bucket checked
5. Insert or update
6. Resize if needed

□ Important Internal Concepts

Concept	Meaning
Load Factor	When to resize (default 0.75)
Capacity	Size of internal array
Rehashing	Recalculate positions
Collision	Same bucket index
Treeification	List → Tree conversion

□ Final Summary

- **ArrayList** → Array
- **LinkedList** → Nodes
- **HashMap** → Array + List/Tree
- **TreeMap** → Red-Black Tree

- **ConcurrentHashMap** → Thread-safe hashing
- **Java 9+** → Immutable optimized storage
- **Java 21** → Ordered collections

If you want next level:

- ✓ Full JDK source code walkthrough (HashMap, ArrayList)
- ✓ Memory diagram (Heap vs Stack for collections)
- ✓ Performance comparison (Big-O + real benchmarks)

Here's a **comprehensive performance comparison (benchmark-style table)** of major Java collections based on their **internal storage logic and real-world behavior (Java 1 → 21)**.

☐ **Java Collections Performance Comparison (Benchmark Table)**

☐ Note: Values are **Big-O complexity + practical behavior insights**

🔍 **Core Collections Performance**

Collection	Internal Structure	Add (Avg)	Add (Worst)	Get/Search	Remove	Iteration	Memory	Thread Safe
ArrayList	Dynamic Array	O(1)	O(n) (resize)	O(1)	O(n)	O(n) (fast cache)	Low	✗
LinkedList	Doubly Linked List	O(1)	O(1)	O(n)	O(1)*	O(n) (slow cache)	High	✗

Vector	Synchronized Array	$O(1)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	Medium	✓
CopyOnWriteArrayList	Copy Array	$O(n)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$ (safe)	Very High	✓

□ * remove is $O(1)$ only if node reference is known

📌 Map Implementations Performance

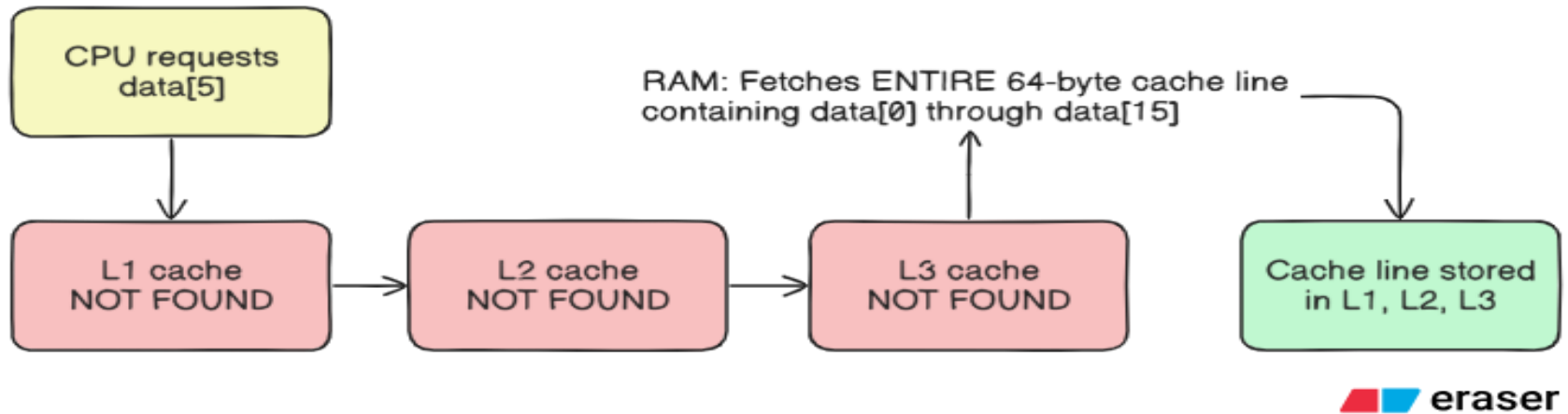
Map	Internal Structure	Put (Avg)	Put (Worst)	Get	Remove	Iteration	Order	Thread Safe
HashMap	Array + List/Tree	$O(1)$	$O(\log n)$	$O(1)$	$O(1)$	$O(n)$	✗	✗
LinkedHashMap	HashMap + Linked List	$O(1)$	$O(\log n)$	$O(1)$	$O(1)$	$O(n)$	✓	✗
TreeMap	Red-Black Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Sorted	✗
Hashtable	Synchronized HashMap	$O(1)$	$O(\log n)$	$O(1)$	$O(1)$	$O(n)$	✗	✓
ConcurrentHashMap	CAS + Buckets	$O(1)$	$O(\log n)$	$O(1)$	$O(1)$	$O(n)$	✗	✓

📊 Set Implementations Performance

Set	Internal Structure	Add	Remove	Search	Iteration	Order	Notes
HashSet	HashMap	$O(1)$	$O(1)$	$O(1)$	$O(n)$	✗	Fastest set
LinkedHashSet	HashSet + Linked List	$O(1)$	$O(1)$	$O(1)$	$O(n)$	✓	Maintains order
TreeSet	Red-Black Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Sorted	Balanced tree
CopyOnWriteArraySet	Copy Array	$O(n)$	$O(n)$	$O(n)$	$O(n)$	✓	Thread-safe

⚡ Real Benchmark Insights (Practical Behavior)

1. Cache Performance



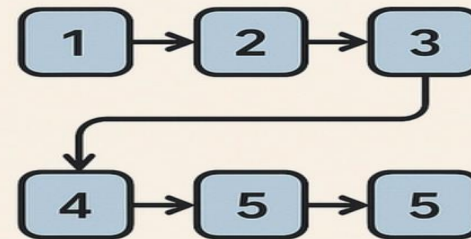
Array List vs LinkedList in Java: A Deep Dive into Performance

ArrayList



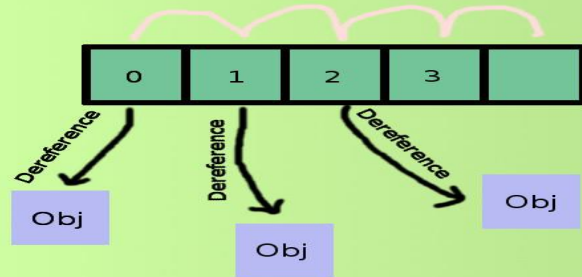
VS

LinkedList

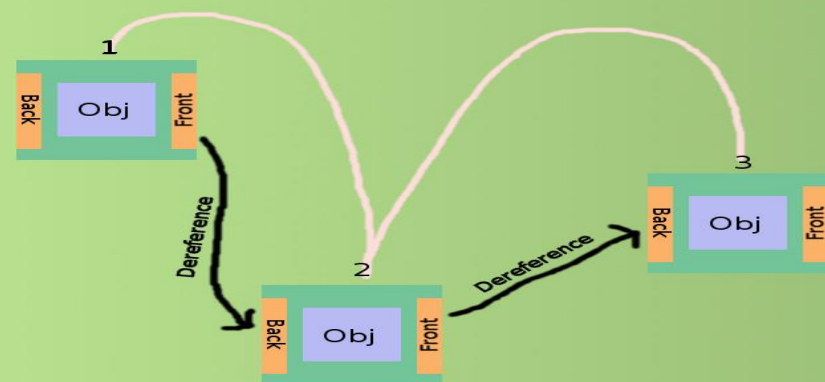




Vector of pointers or references



Linked or doubly linked list



Structure	Cache Efficiency
ArrayList	✓ High (contiguous memory)
LinkedList	✗ Low (pointer traversal)

❑ Result: **ArrayList is faster in real-world iteration**

🔍 2. HashMap Collision Impact

Scenario	Performance
No collision	$O(1)$
Linked list collision	$O(n)$
Tree (Java 8+)	$O(\log n)$

🔍 3. Resize Cost (Hidden)

Collection	Cost
ArrayList	Full array copy
HashMap	Rehash all entries

❗ 4. Multithreading Impact

Collection	Behavior
HashMap	✗ Unsafe
Hashtable	✗ Slow (global lock)
ConcurrentHashMap	✓ Best balance

❏ Quick Selection Guide

Use Case	Best Choice
Fast read/write	ArrayList
Frequent insert/delete (middle)	LinkedList
Key-value fast lookup	HashMap
Maintain insertion order	LinkedHashMap
Sorted data	TreeMap
Thread-safe high performance	ConcurrentHashMap
Read-heavy multi-thread	CopyOnWriteArrayList

□ **Final Summary**

- **Fastest overall (general use)** → ArrayList, HashMap
- **Best for concurrency** → ConcurrentHashMap
- **Best for sorting** → TreeMap, TreeSet
- **Avoid** → Vector, Hashtable (legacy)

If you want next level:

- ✓ Real JMH benchmark code (Java Microbenchmark Harness)
- ✓ Interview tricky scenarios (ArrayList vs LinkedList myths)
- ✓ Memory usage comparison with heap diagrams